

同济大学计算机系

数字逻辑课程综合实验报告



学 号 2351520

姓 名 毛星博

专 业 基础学科拔尖基地(计算机)

授课老师 张冬冬

一、实验内容

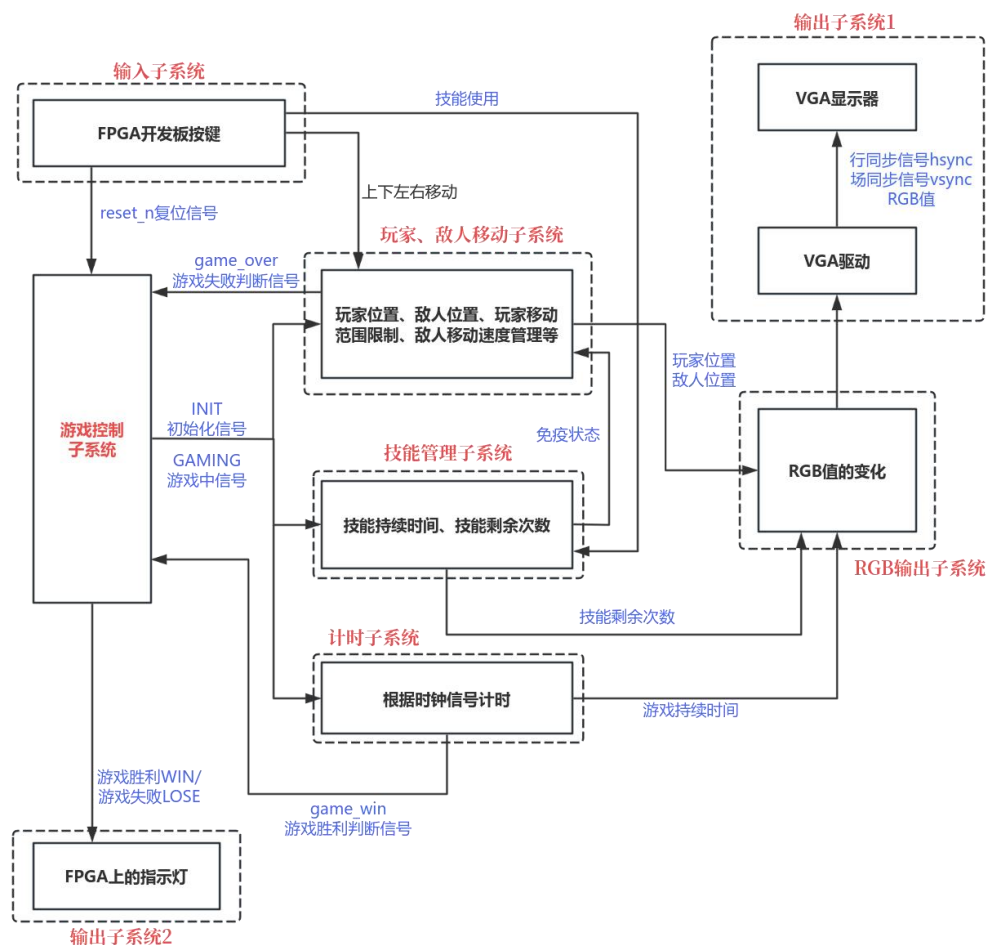
基于 verilog 语言,使用 VGA 显示器,在可编程 FPGA 开发板上实现游戏“大逃杀”。戏通过开发板上的按键控制,游戏画面显示在 VGA 显示器上。

游戏规则:

- 玩家控制代表角色的绿色方块进行移动
- 需要躲避不断移动红色敌人
- 撞到障碍物即游戏结束
- 障碍物的速度会随着每次折返(碰到墙壁反弹)而加快(有最大速度)
- 屏幕上方显示总游戏时间,当坚持到 30 秒即通关
- 玩家有 3 次使用技能的机会,当按下技能按键时,玩家会进入免疫状态,此时碰到敌人不会导致游戏失败。(技能剩余次数会显示在屏幕右下角)

二、“大逃杀”游戏:数字系统总框图

(按由顶向下方法进行子系统的划分,给出包含各子系统相互关系及控制信号的总框图,并对各子系统功能及实现进行概述。具体可参考教材 183 页的相关描述方法。)



1.输入子系统:

功能说明：通过 FPGA 开发板按键实现用户输入控制

2.玩家、敌人移动控制子系统:

功能说明:

- 控制玩家角色移动（上下左右）
- 管理移动方块的自动移动
- 处理边界碰撞检测
- 控制移动速度变化

实现方法:

- 使用独立模块 `move_sheep.v` 实现玩家移动控制
- 采用时序逻辑实现按键响应
- 在 `frame_flag` 同步信号下更新位置
- 使用参数化设计定义移动速度和边界
- 位置坐标使用 11 位寄存器存储
- 通过组合逻辑实现边界检测

主要参数:

- 玩家移动速度：5 像素/帧
- 方块初始速度：1 像素/帧
- 方块最大速度：15 像素/帧
- 移动边界：距离屏幕边缘 80 像素

3.技能管理子系统:

功能说明:

- 响应技能触发按键
- 管理技能使用次数（3 次）
- 控制技能免疫状态
- 显示剩余技能次数

实现方法:

- 使用 `skill_controller.v` 模块实现
- 采用状态机设计：
 - IDLE：等待技能触发
 - ACTIVE：技能生效状态
- 使用 32 位计数器实现技能持续时间
- 使用 2 位计数器记录技能次数
- 七段数码管显示技能剩余次数

主要参数:

- 技能持续时间：3 秒
- 免疫效果：碰撞无效

4.计时子系统:

功能说明:

- 游戏时间计数（30 秒目标）
- 游戏胜利条件判定

- 生成游戏胜利信号

实现方法：

- 集成在 game_controller.v 中实现
- 使用 26 位计数器实现 1 秒计时（65MHz 时钟）
- 使用 16 位计数器记录游戏时间
- 采用组合逻辑判断胜利条件
- 通过 display_mode 信号控制显示状态

计时参数：

- 计时单位：1 秒
- 胜利条件：存活 30 秒

5.RGB 输出子系统：

功能说明：

- 生成游戏画面显示数据
- 实现不同显示效果：
 - 正常模式：显示玩家（绿色）和方块（红色）
 - 免疫状态：玩家显示特效（闪烁）
 - 游戏结束：红色条纹效果
 - 游戏胜利：全屏绿色
- 显示游戏时间和技能次数

实现方法：

- 在 mxb_rgb.v 中实现
- 使用组合逻辑判断像素位置
- 通过 case 语句实现不同显示模式：
 - 4'd0：正常游戏画面
 - 4'd1：游戏结束画面
 - 4'd2：游戏胜利画面
- 使用计数器实现免疫状态闪烁效果
- 采用参数化设计定义显示位置和大小

显示参数：

- 分辨率：1024x768
- 颜色深度：4 位/通道（RGB444）

6.输出子系统 1（VGA 显示子系统）：

功能说明：

- 生成 VGA 时序信号
- 输出 RGB 显示数据

时序参数：

- 像素时钟：65MHz
- 分辨率：1024x768
- 频率：60Hz

7.输出子系统 2（LED 灯指示游戏胜利或失败）：

功能说明： LED 指示灯显示游戏状态

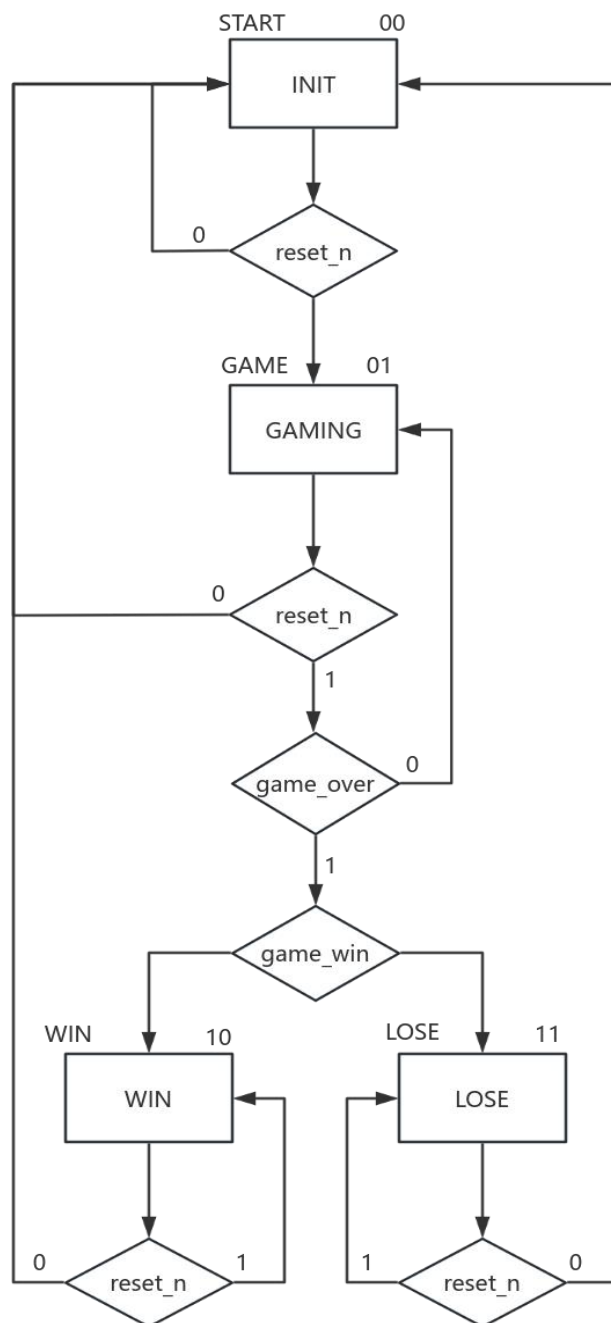
三、系统控制器设计

（要求画出所设计数字系统的 ASM 流程图，列出状态转移真值表。由状态转移真值表，求出系统控制器的次态激励函数表达式和控制命令逻辑表达式，并用 Logisim 画出系统控制器逻辑方案图。）

一共 4 个状态 **START**、**GAME**、**WIN**、**LOSE**。采用计时器法，使用两个 D 触发器 A、B，对四个状态分别编码为 00、01、10、11。

控制命令：**INIT**、**GAMING**、**WIN**、**LOSE**

ASM 流程图：



状态转移真值表：

PS（现态）			NS（次态）			转移条件
状态	A	B	状态	A	B	C
INIT	0	0	INIT	0	0	reset_n=0
INIT	0	0	GAME	0	1	reset_n=1
GAME	0	1	INIT	0	0	reset_n=0
GAME	0	1	GAME	0	1	reset_n=1,game_over=0
GAME	0	1	WIN	1	0	reset_n=1,game_over=1,game_win=1
GAME	0	1	LOSE	1	1	reset_n=1,game_over=1,game_win=0
WIN	1	0	WIN	1	0	reset_n=1
WIN	1	0	INIT	0	0	reset_n=0
LOSE	1	1	LOSE	1	1	reset_n=1
LOSE	1	1	INIT	0	0	reset_n=0

状态转移方程组：

$D1 = \overline{A} B \text{ reset_n game_over} + A \text{ reset_n}$

$D2 = (A \odot B) \text{ reset_n} + \overline{A} B \overline{\text{reset_n game_over game_win}}$

控制命令输出：

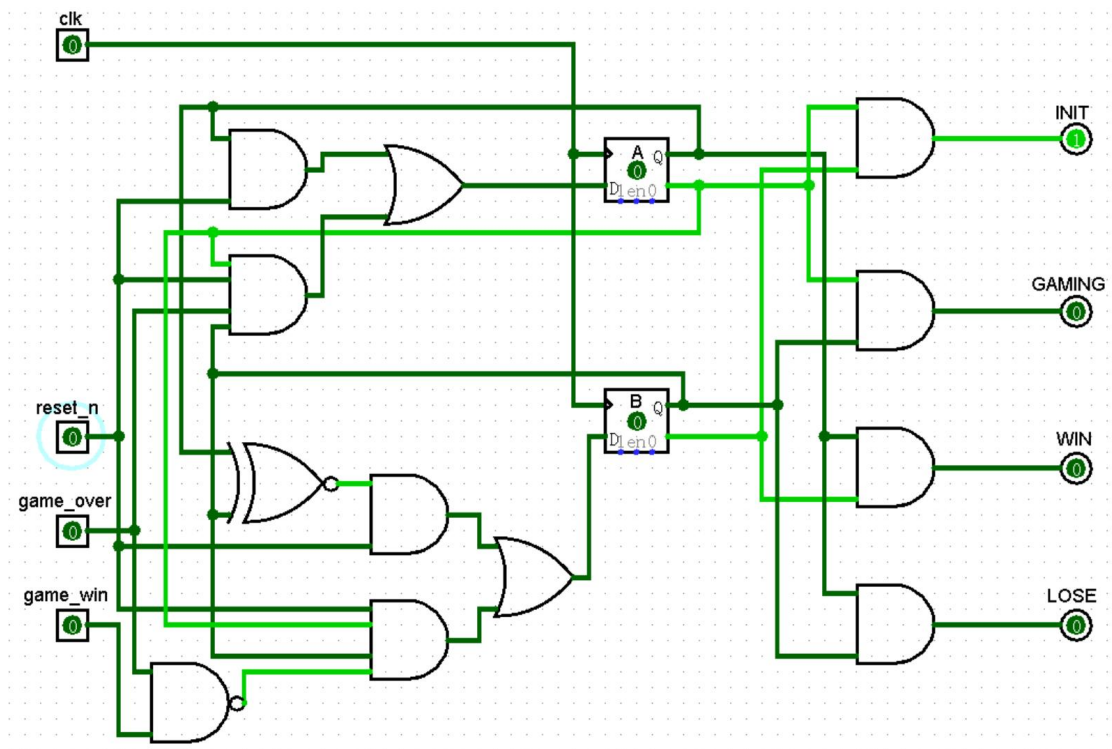
$\text{INIT} = \overline{A} \overline{B}$

$\text{GAMING} = \overline{A} B$

$\text{WIN} = A \overline{B}$

$\text{LOSE} = A B$

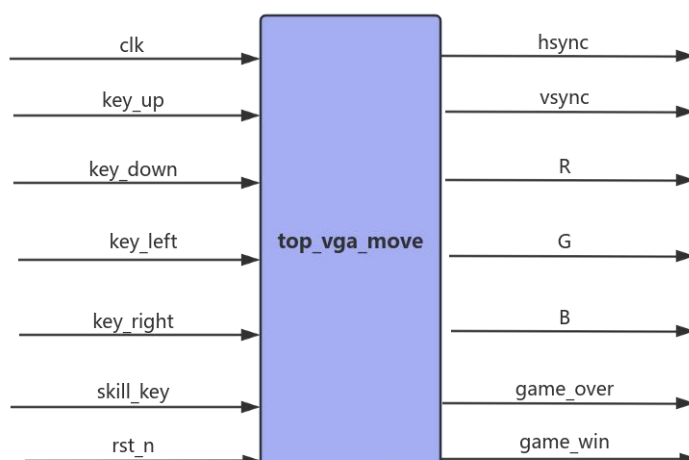
logisim 原理图：

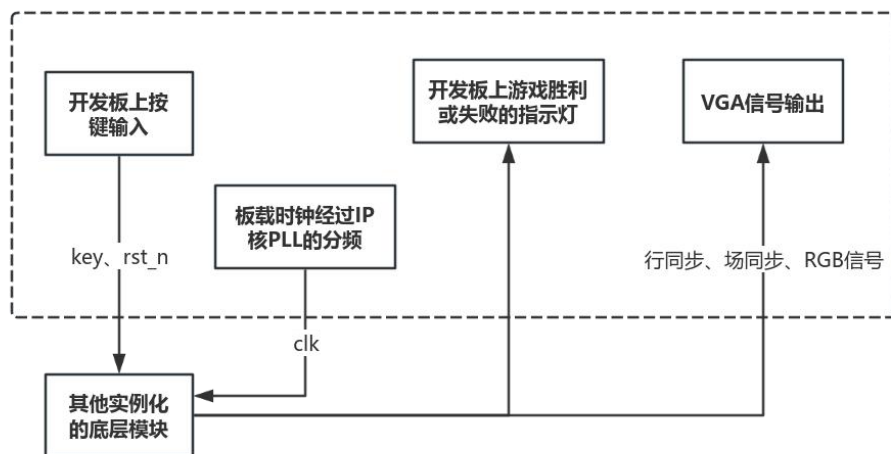


四、子系统模块建模

（该部分要求对实验中的所有子系统模块进行描述，给出各子系统的功能框图及接口信号定义，并列出各模块建模，图文描述，描述清楚设计及实现的思路，不要在此处放 verilog 代码，所有的 verilog 代码在附录部分）

①顶层模块 **module top_vga_move**





接口信号定义:

时钟和复位

- clk: 由 IP 核将系统时钟的 100MHz 转为 65MHz
- rst_n: 输入, 低电平复位信号

控制输入

- key_up: 输入, 向上移动按键
- key_down: 输入, 向下移动按键
- key_left: 输入, 向左移动按键
- key_right: 输入, 向右移动按键
- skill_key: 输入, 技能触发按键

VGA 显示输出

- hsync: 输出, VGA 水平同步信号
- vsync: 输出, VGA 垂直同步信号
- R[3:0]: 输出, 红色分量
- G[3:0]: 输出, 绿色分量
- B[3:0]: 输出, 蓝色分量

游戏状态输出

- game_over: 输出, 游戏结束信号
- game_win: 输出, 游戏胜利信号

功能描述:

时钟管理:

- 通过 Clocking Wizard 将 100MHz 系统时钟转换为 65MHz VGA 时钟
- 监控 PLL 锁定状态, 确保时钟稳定

模块实例化与互连:

- 实例化并连接所有子模块:
- 移动控制模块(move_sheep)
- VGA 显示模块(mxb_rgb)
- 游戏控制器(game_controller)
- 技能控制器(skill_controller)

复位控制:

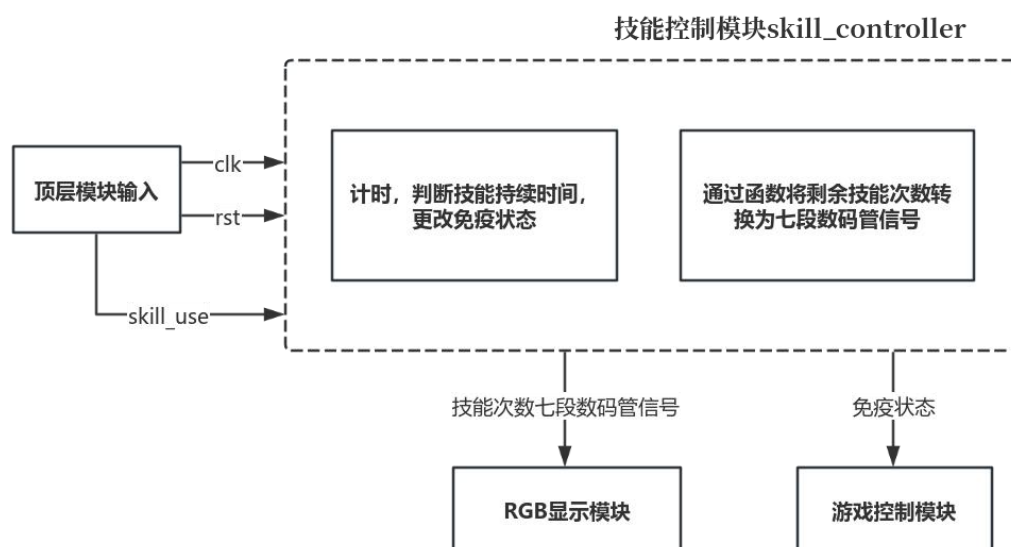
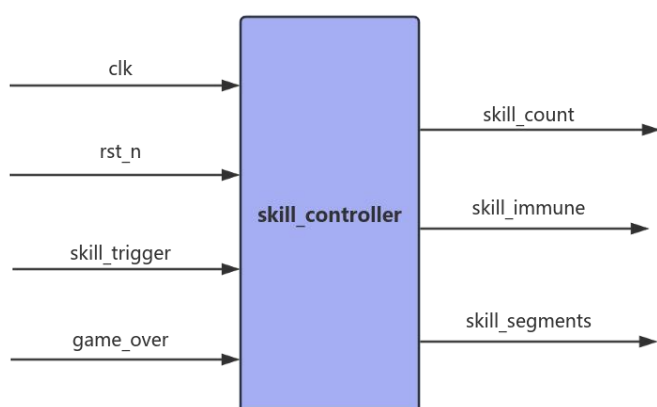
- 所有子模块的复位信号都与 PLL 锁定信号相与
- 确保系统在时钟稳定后才开始工作

实现思路：

顶层模块是整个系统的指挥者，负责协调各个功能模块的工作。首先通过 PLL 将 100MHz 的系统时钟转换为 65MHz 的 VGA 时钟，这个频率是经过计算的，确保满足 1024×768、60Hz 显示要求。

模块间的互联是重点，需要正确连接各类控制信号和状态信号。所有子模块的复位信号都与 PLL 锁定信号相与，确保系统在时钟稳定后才开始工作。按键输入经过处理后分发给相应模块，显示输出的控制也在这里统一管理。在设计中特别注意了时序约束，确保关键信号的稳定传输。整个设计采用层次化的模块结构，既保证了功能的完整性，又便于后期维护和功能扩展。模块化的设计思路也便于后期的调试和优化。

②技能控制模块 skill_controller



接口信号定义：

时钟和控制

- clk: 由 IP 核将系统时钟的 100MHz 转为 65MHz
- reset_n: 低电平复位
- skill_trigger: 技能触发
- game_over: 游戏结束

输出状态

- skill_count[1:0]: 剩余技能次数
- skill_immune: 技能免疫状态
- skill_segments[6:0]: 七段显示信号

功能描述：

技能管理：

- 初始 3 次技能使用机会
- 技能触发时启动免疫计时
- 免疫持续 3 秒

状态显示：

- 通过七段数码管显示剩余次数
- 更新显示信号

计时控制：

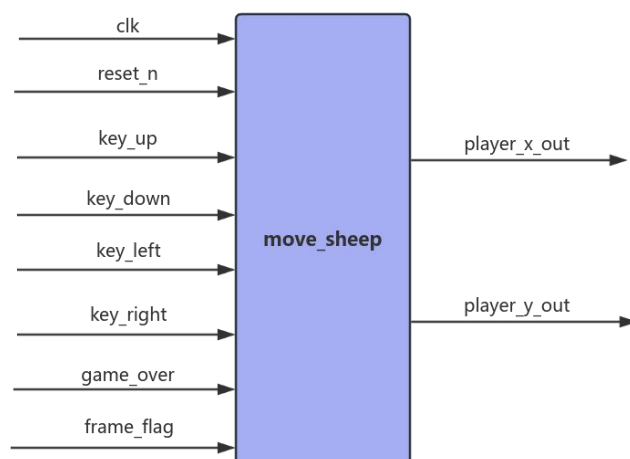
- 技能免疫时间计数
- 自动解除免疫状态

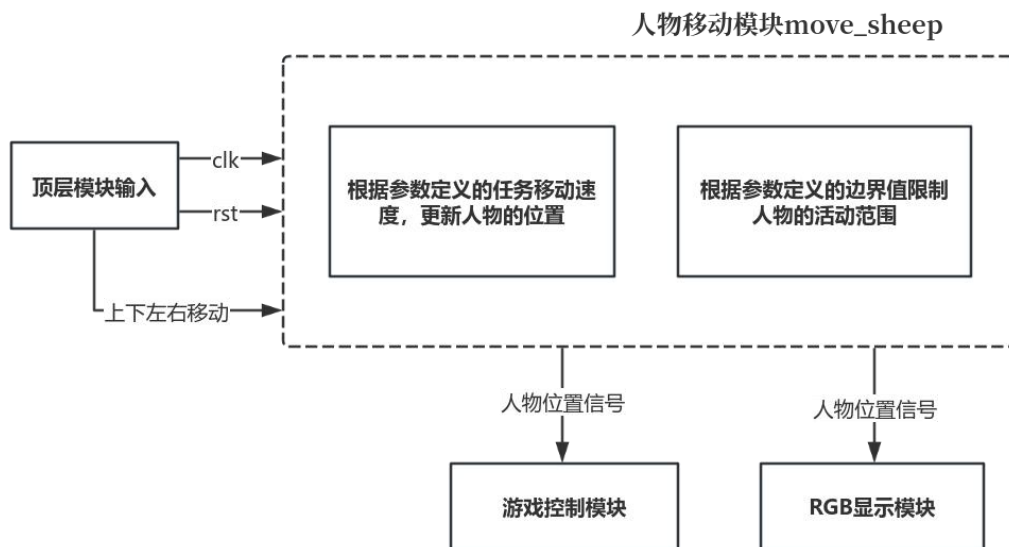
实现思路：

这个模块实现了游戏的技能系统。玩家初始拥有 3 次技能使用机会，每次使用后进入 3 秒的免疫状态。

实现上使用计数器精确控制免疫时间，当玩家触发技能时，计数器开始从预设值递减，直到归零才解除免疫状态。剩余技能次数通过七段数码管实时显示。在模块内部将剩余技能次数转化为七段数码管信号再传给 RGB 显示模块。

③ 人物移动模块 move_sheep





接口信号定义:

时钟和复位

- clk: 由 IP 核将系统时钟的 100MHz 转为 65MHz
- reset_n: 低电平复位信号

输入控制信号

- key_up: 向上移动按键
- key_down: 向下移动按键
- key_left: 向左移动按键
- key_right: 向右移动按键
- game_over: 游戏结束信号
- frame_flag: 帧同步标志

输出位置信号

- player_x_out[10:0]: 玩家 X 坐标
- player_y_out[10:0]: 玩家 Y 坐标

功能描述:

位置初始化:

- 复位时将玩家位置初始化到屏幕中心
- 初始 X 坐标 = $H_ACTIVE/2$
- 初始 Y 坐标 = $V_ACTIVE/2$

移动控制:

- 每个帧同步时刻检测按键状态
- 根据按键输入更新位置坐标
- 移动速度固定为 $PLAYER_SPEED$

边界检测:

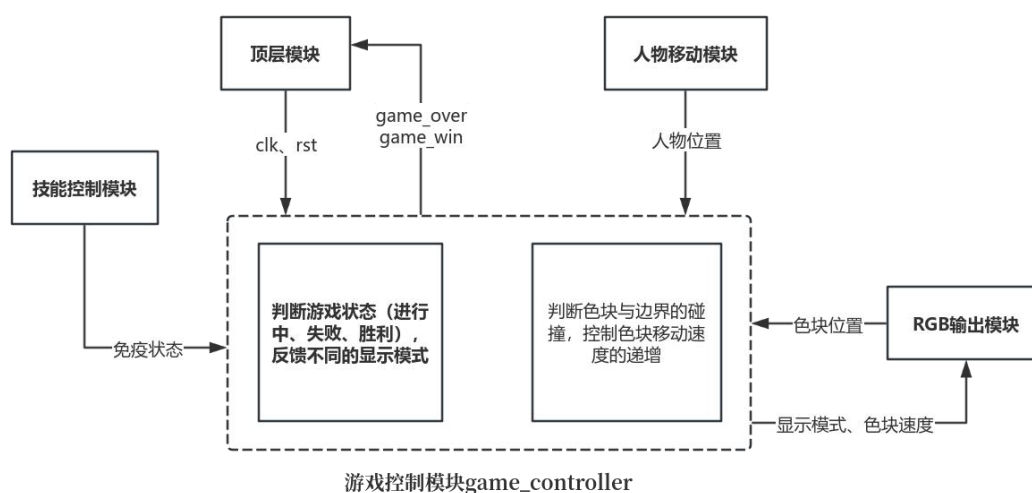
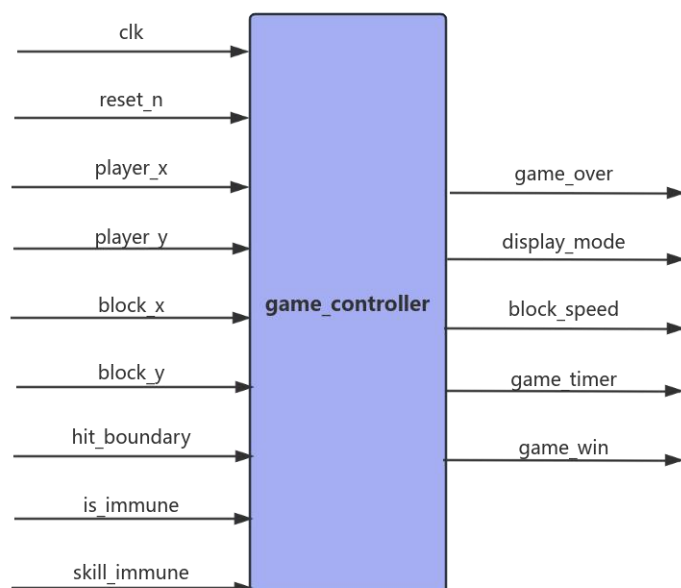
- 确保玩家不会移出显示边界
- 上边界: $BORDER_MARGIN + PLAYER_SPEED$
- 下边界: $V_ACTIVE - BORDER_MARGIN - PLAYER_SIZE - PLAYER_SPEED$

- 左边界: $BORDER_MARGIN + PLAYER_SPEED$
- 右边界: $H_ACTIVE - BORDER_MARGIN - PLAYER_SIZE - PLAYER_SPEED$

实现思路:

这个模块处理玩家角色的移动控制。采用同步时序设计，在每个帧同步时刻检测四个方向键的输入状态。移动速度通过 `PLAYER_SPEED` 参数控制，可以根据需要调整。为了实现平滑的移动效果，在每次移动时都进行边界检测，确保玩家不会移出显示区域。边界检测考虑了多个因素：边界距离(`BORDER_MARGIN`)、玩家大小(`PLAYER_SIZE`)和移动速度(`PLAYER_SPEED`)。每次移动前都会计算新位置是否超出边界，只有在安全范围内才允许移动。在游戏结束状态下，模块会忽略所有按键输入，保持玩家位置不变。

④ 游戏控制模块 `game_controller`



接口信号定义:

时钟和复位

- `clk`: 系统时钟,65MHz
- `reset_n`: 低电平复位信号

位置输入信号

- `player_x[10:0]`: 玩家 X 坐标
- `player_y[10:0]`: 玩家 Y 坐标
- `block_x[10:0]`: 方块 X 坐标
- `block_y[10:0]`: 方块 Y 坐标
- `hit_boundary`: 方块碰到边界信号
- `is_immune`: 开局免疫状态
- `skill_immune`: 技能免疫状态

输出控制信号

- `game_over`: 游戏结束信号
- `display_mode[3:0]`: 显示模式控制
- `block_speed[7:0]`: 方块移动速度
- `game_timer[15:0]`: 游戏计时器
- `game_win`: 游戏胜利信号

功能描述:

碰撞检测:

- 检测玩家与移动方块的碰撞
- 考虑免疫状态的影响
- 触发游戏结束条件

速度控制:

- 方块每次碰到边界后速度增加
- 速度上限为 `MAX_SPEED`
- 初始速度为 `BASE_SPEED`

游戏状态管理:

- 计时器更新(每秒更新)
- 胜利条件判定(生存 30 秒)
- 显示模式切换

实现思路:

这个模块是整个游戏的大脑，负责处理游戏规则和状态管理。首先，碰撞检测通过比较玩家和移动方块的坐标范围来判断是否发生碰撞。考虑到游戏的趣味性，我实现了免疫机制，在使用技能时可以暂时无视碰撞。同时为了使玩家适应游戏，不至于在游戏开始就死掉，设置了进入游戏免疫 5 秒的机制。

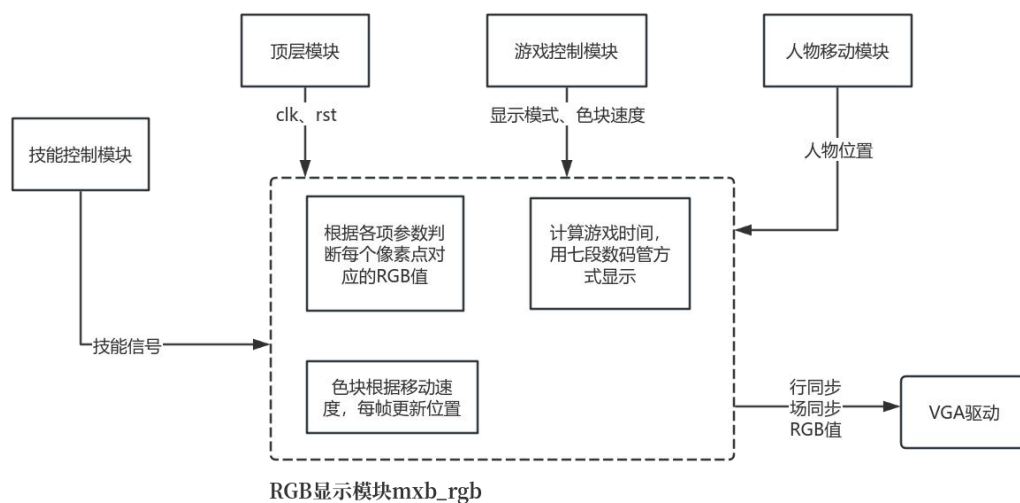
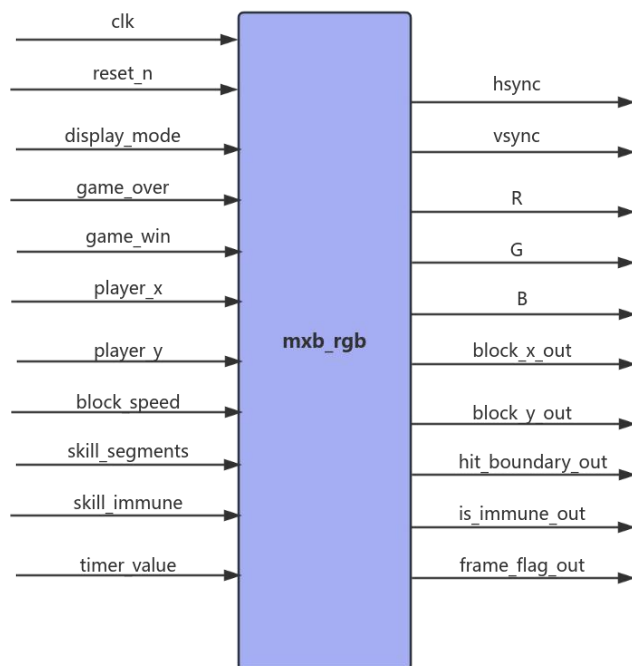
游戏难度设计采用渐进式增加：移动方块每次碰到边界后速度都会增加，但设置了最大速度限制，避免难度过高。

计时器记录游戏时间，每秒更新一次，达到 30 秒即触发胜利条件。时间控制子系统集成在模块内部，通过函数计算出时间对应的七段数码管信号并传给 RGB 显示模块。

状态管理采用组合逻辑和时序逻辑相结合的方式，游戏可能处于正常、结束

或胜利三种状态，每种状态都有对应的显示模式和控制逻辑。

⑤ RGB 显示模块 mxb_rgb



接口信号定义:

时钟和同步信号

- clk: 系统时钟
- reset_n: 低电平复位
- hsync: 行同步输出
- vsync: 场同步输出

颜色输出

- R[3:0]: 红色分量
- G[3:0]: 绿色分量

- B[3:0]: 蓝色分量

游戏状态输入

- display_mode[3:0]: 显示模式
- game_over: 游戏结束标志
- game_win: 游戏胜利标志

位置和状态信号

- player_x/y[10:0]: 玩家位置
- block_x/y_out[10:0]: 方块位置
- block_speed[7:0]: 方块速度
- skill_segments[6:0]: 技能显示
- timer_value[15:0]: 计时器值

功能描述:

图形渲染:

- 绘制玩家角色(绿色方块)
- 绘制移动方块(红色)
- 绘制边界(黄色)

状态显示:

- 显示计时器数值
- 显示技能剩余次数
- 显示游戏状态(结束/胜利)

特效处理:

- 免疫状态闪烁效果
- 游戏结束红色条纹
- 游戏胜利全绿显示

实现思路:

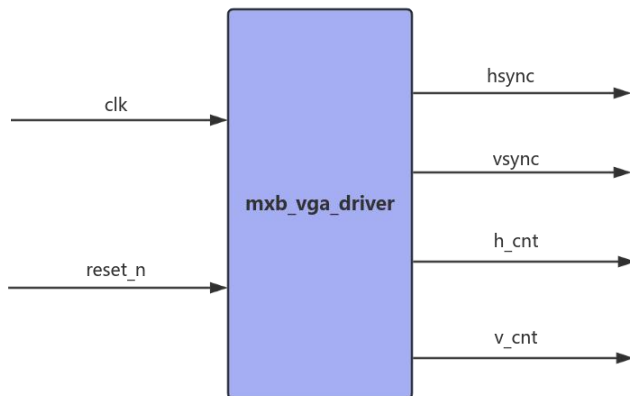
这个模块是整个游戏画面渲染的核心。我设计了一个优先级显示机制，通过判断当前扫描位置是否在各个元素的显示范围内来决定输出的 RGB 值。显示优先级从高到低依次是：玩家角色、移动方块、边界线、计时器显示、技能显示和背景。

在实现中，玩家角色正常显示为绿色，但在免疫状态下会闪烁显示，通过计数器控制颜色交替来实现闪烁效果。移动方块显示为红色，随着游戏进行会逐渐加快移动速度。边界线采用黄色显示，清晰地划分出游戏区域。

计时器和技能次数的显示采用七段数码管的方式，我实现了一个数字显示函数，可以将数值转换为对应的段选信号。游戏状态的显示也很关键，游戏结束时通过红色条纹效果提示玩家，胜利时则全屏显示绿色。

为了避免画面撕裂，使用 frame_flag 控制画面更新，确保在帧同步时刻进行状态更新。

⑥ VGA 驱动模块 mxv_vga_driver



接口信号定义：

时钟和复位

- **clk**: 由 IP 核将系统时钟的 100MHz 转为 65MHz
- **reset_n**: 低电平复位信号

同步信号输出

- **hsync**: 行同步信号输出
- **vsync**: 场同步信号输出

计数器输出

- **h_cnt[10:0]**: 行扫描计数器
- **v_cnt[10:0]**: 场扫描计数器

功能描述：

行扫描控制：行计数器(**h_cnt**)从 0 计数到 1343，每个时钟周期计数值加 1，到达 **H_TOTAL-1** 时重置为 0。

生成行同步信号(**hsync**)：在 **h_cnt = 0** 到 **H_SYNC-1** 期间为高电平，其余时间为低电平。

场扫描控制：场计数器(**v_cnt**)从 0 计数到 805，每当 **h_cnt** 达到最大值时 **v_cnt** 加 1，到达 **V_TOTAL-1** 时重置为 0。

生成场同步信号(**vsync**)：在 **v_cnt = 0** 到 **V_SYNC-1** 期间为高电平，其余时间为低电平。

实现思路：

这个模块采用双计数器结构实现 VGA 时序控制。行计数器从 0 计数到 1343，场计数器从 0 计数到 805，分别对应水平和垂直扫描。根据 VGA 标准时序要求，我将显示区域分为同步、后沿、有效显示和前沿四个部分。**hsync** 信号在行计数值 0 到 135 期间有效，**vsync** 信号在场计数值 0 到 5 期间有效。有效显示区域是 1024×768，这部分是实际显示图像的区域。

为了提高模块的可复用性，所有时序参数都定义为可配置的参数。同步信号的极性也通过参数控制，可以适应不同型号显示器的要求。在实现时，特别注意了时序的稳定性，确保计数器在时钟上升沿更新，同步信号的翻转也按照时序要求进行。

五、测试模块建模

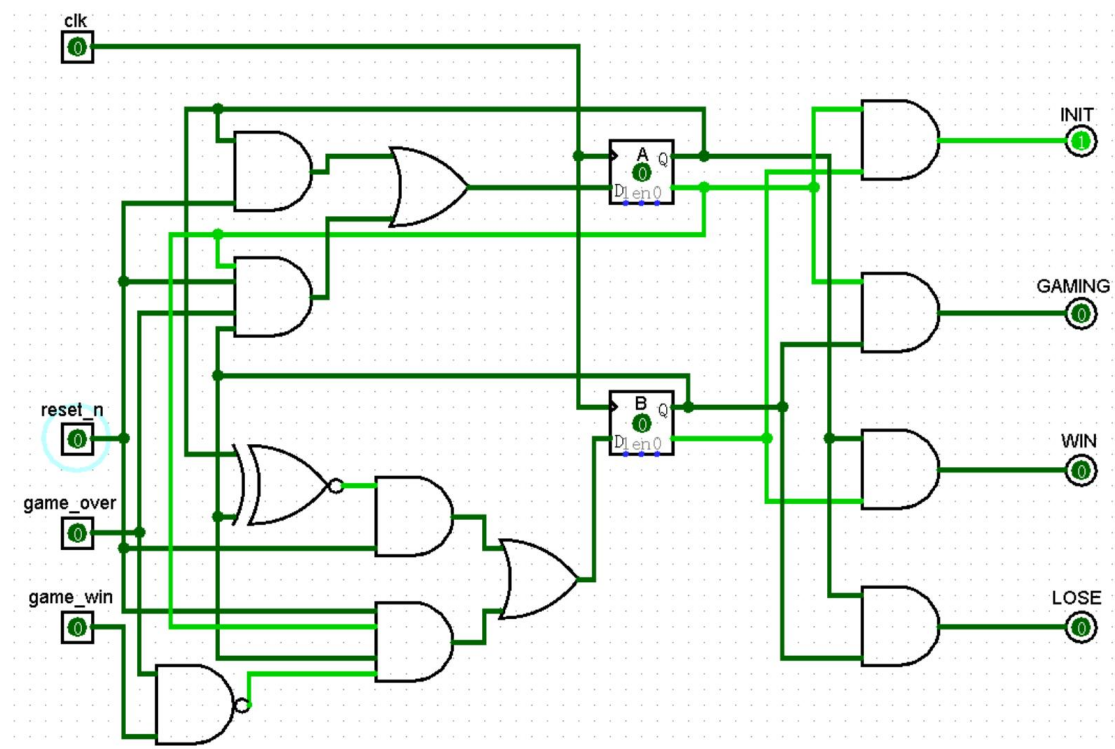
（要求列写各建模模块的 test bench 模块测试逻辑，不要在此处放 verilog 代码，所有的 verilog 代码在附录部分）

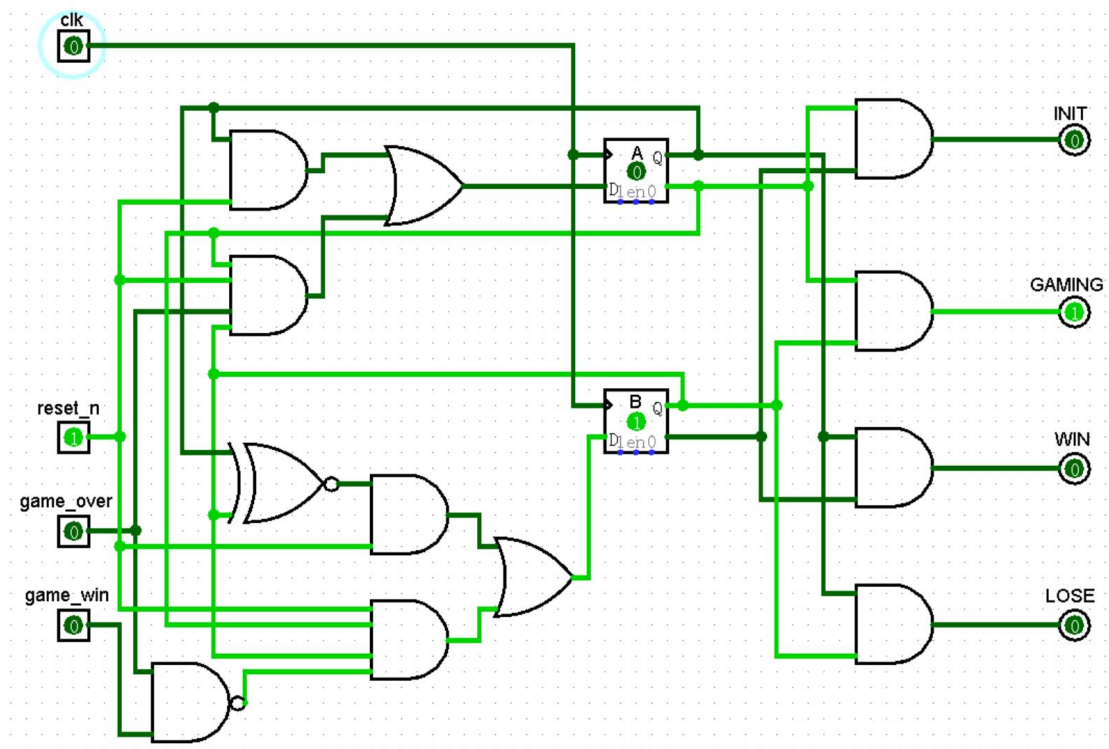
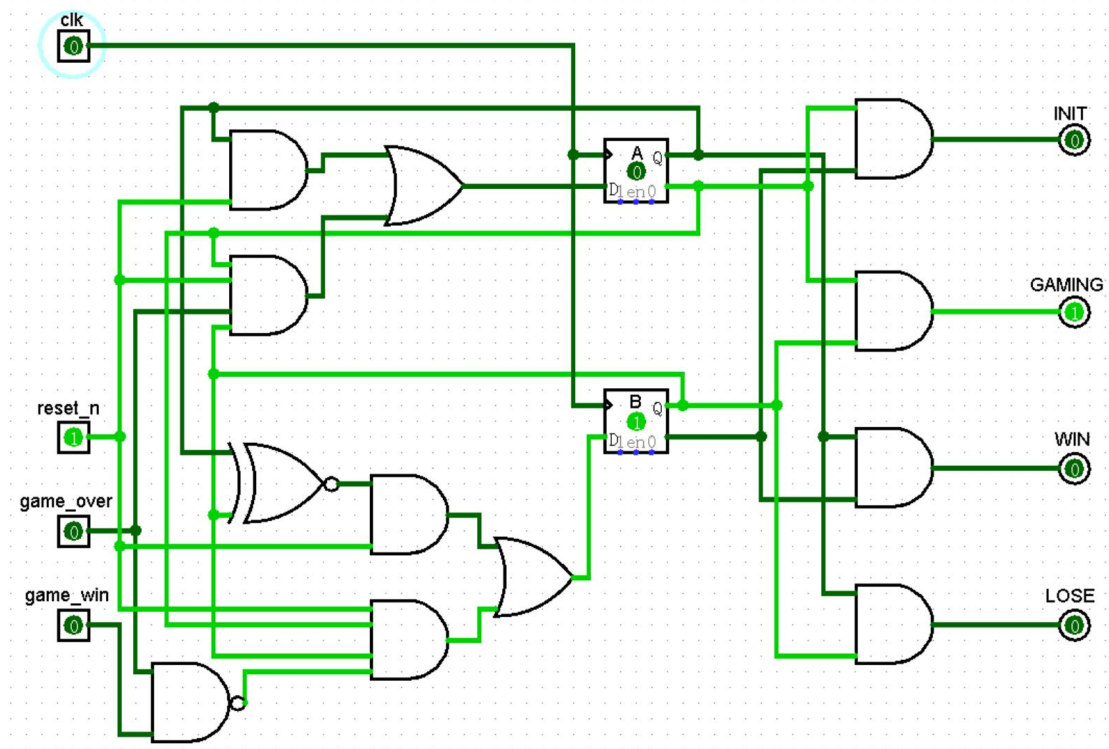
本实验没有 testbench，通过下板后观察 VGA 显示器的内容进行测试

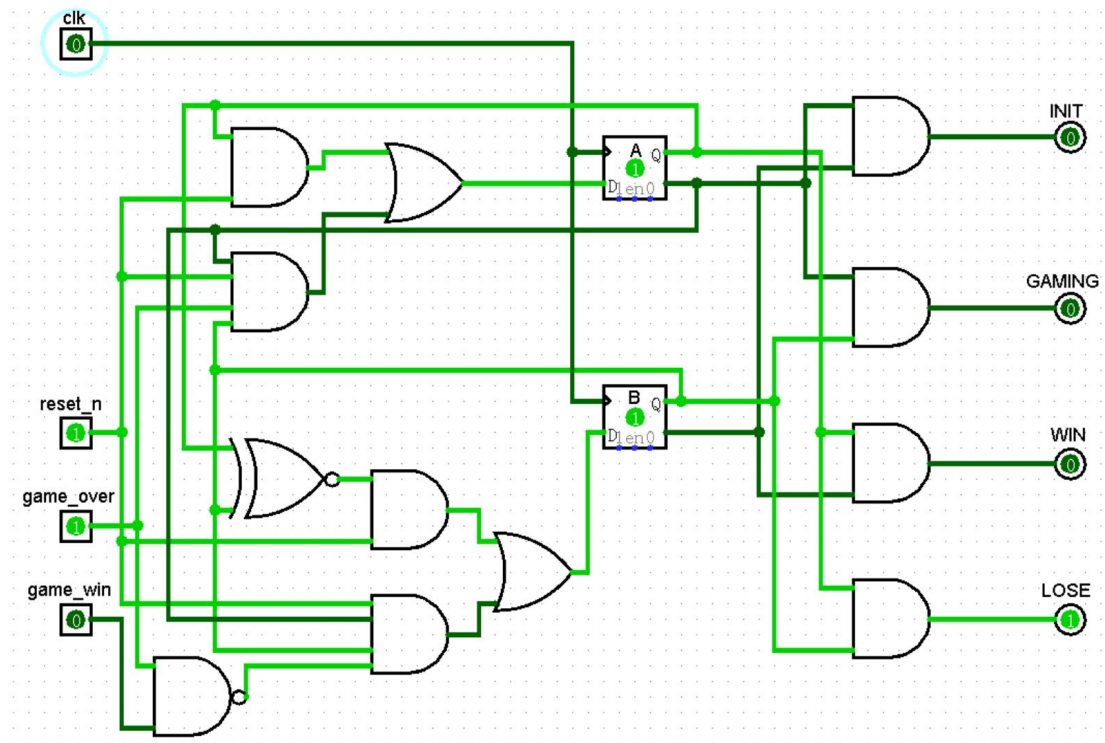
六、实验结果

（该部分可截图说明，可包含 logisim 逻辑验证图、modelsim 仿真波形图、以及下板后的实验结果贴图）

Logisim 逻辑验证（控制器部分）

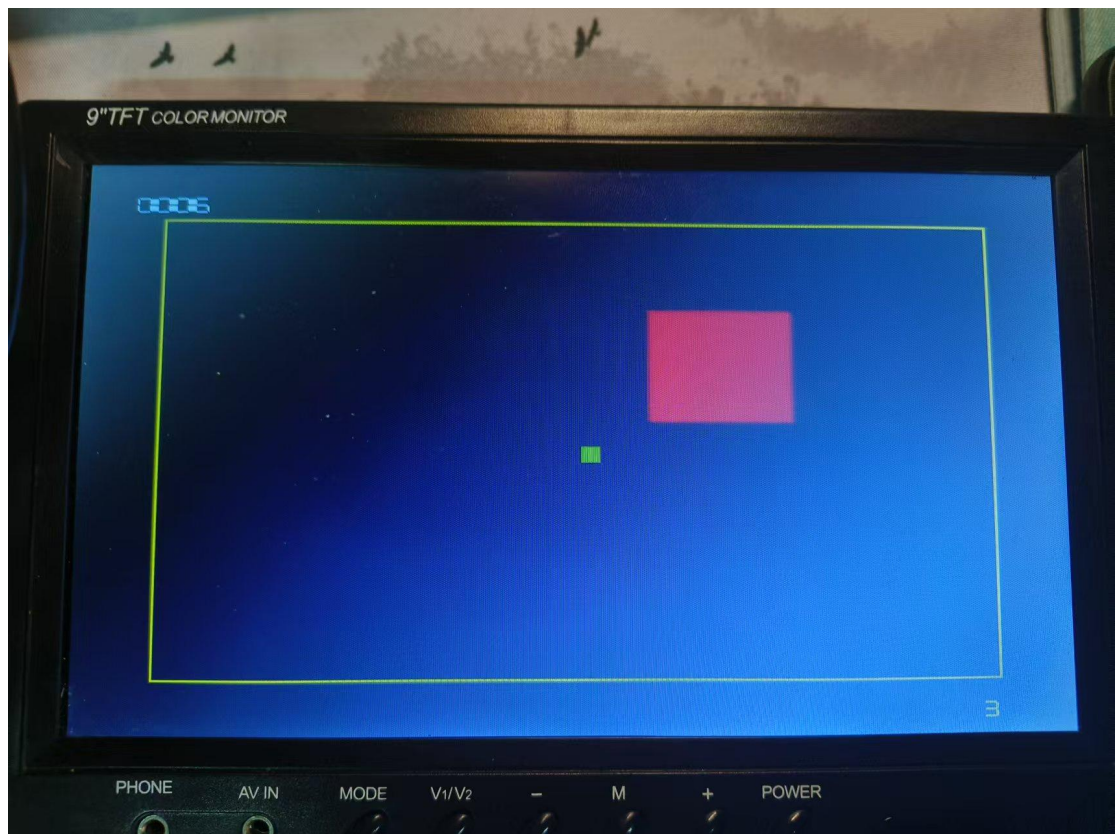




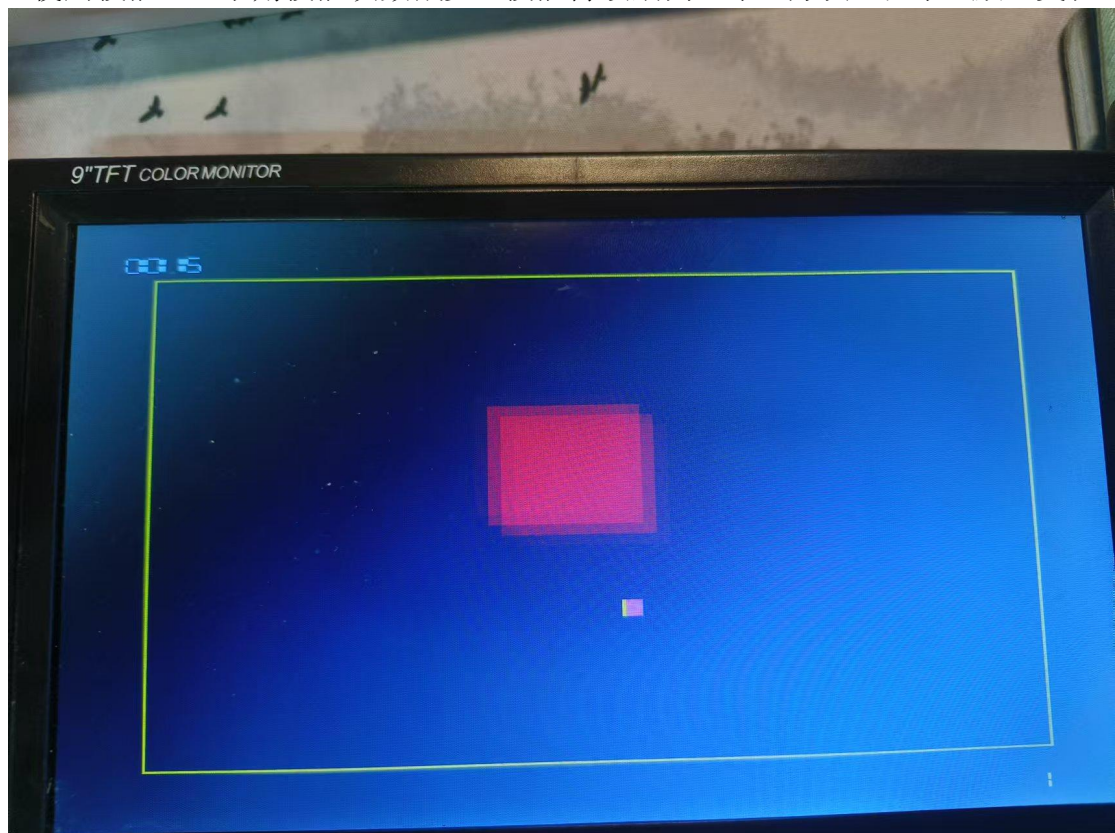


下板实验结果:

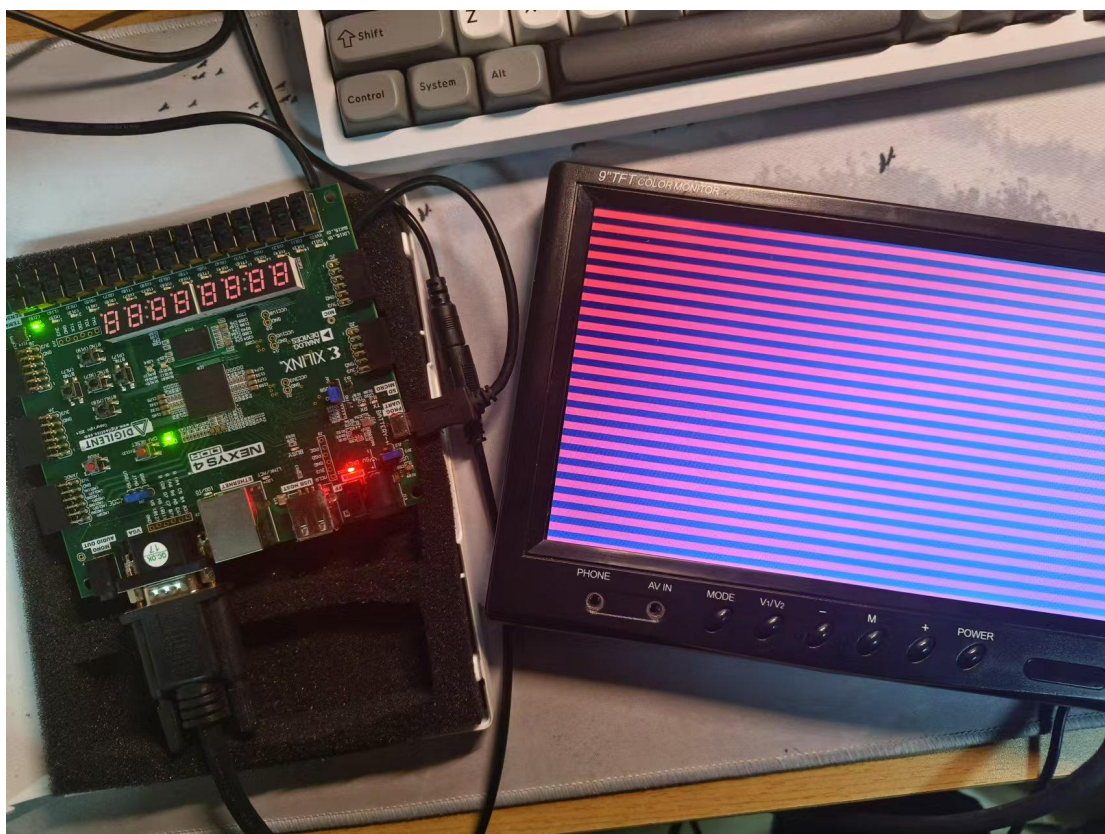
1.游戏界面（左上角显示持续时间，右下角显示剩余技能次数）



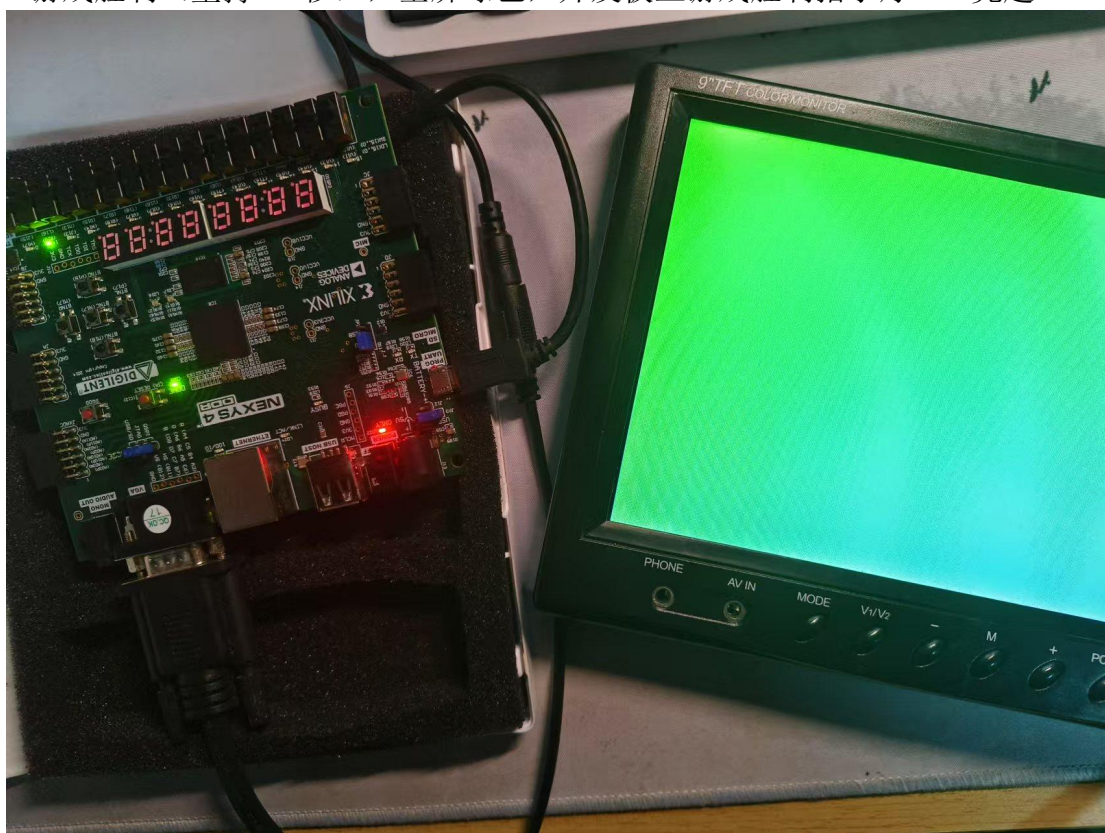
2.使用技能（左下角技能次数减少，技能持续期间，绿色方块（玩家）颜色变化）



3.游戏失败，全屏红色条纹，开发板上游戏失败指示灯 H17 亮起



4.游戏胜利（坚持 30 秒），全屏绿色，开发板上游戏胜利指示灯 J15 亮起



七、结论

本次实验中，我利用 FPGA 开发板和 VGA 显示器实现了一个名叫"大逃杀"的小游戏。通过对 VGA 协议的深入研究，成功实现了 1024×768 、60Hz 的显示输出。

游戏的核心玩法是控制一个角色躲避不断移动红色方块，通过按键实现上下左右移动，还加入了临时免疫的技能系统增加游戏策略性。

在实现过程中，我采用模块化的设计思路，将整个系统分为 VGA 驱动、显示控制、移动控制、游戏控制和技能控制等模块。通过 vivado 的 IP 核 PLL 实现了时钟管理，将 100MHz 系统时钟转换为 65MHz 的 VGA 时钟。

游戏难度设计采用渐进式增加，方块的移动速度会随着时间推移而加快，玩家需要在 30 秒内存活下来才能获得胜利。

虽然基本功能都已实现，但还有一些遗憾的地方。比如原本我选的部件还有 MP3，计划加入更丰富的音效系统，让不同状态都有相应的音效提示，比如游戏胜利、游戏失败、碰撞边界的音效；开始我还有加入菜单栏的想法，可以选择不同的游戏难度，加入更多“追杀”色块，更快的移动速度和可变的技能次数，但是由于实现难度的问题都暂时放弃了。

八、心得体会及建议

这次项目让我深刻体会到硬件编程和软件编程的巨大差异。记得在刚开始写 verilog 代码时，我的思路还是和写 c++、python 等语言的思路一样，一步一步的执行下来。但是这样根本没法实现功能。搞了半天我才知道，原来硬件描述语言最重要的思想是时序逻辑和并行思维。

最让我印象深刻的是调试 VGA 显示的过程。刚开始时，屏幕上总是显示一些混乱的图案，我花了整整一天时间研究 VGA 时序，在网上查找资料，在 CSDN 找教学，用示波器观察同步信号，最终才理解了前沿、后沿、有效显示区域这些概念的实际含义。这个过程虽然痛苦，但收获很大。现在回想起来，那些折磨人的调试经历反而成了最宝贵的学习经验。

在实现技能系统时也遇到了有趣的问题。最初我设计的免疫效果会导致画面闪烁，后来发现是因为状态更新和显示更新的时序不同步。通过添加帧同步信号才解决了这个问题。这让我明白了在硬件设计中，看似简单的功能背后可能隐藏着复杂的时序问题。同时也学会了使用帧同步这种技巧来协调不同模块间的工作。

游戏控制器的设计让我体会到了状态机的重要性。起初我用简单的 if-else 来处理游戏状态，结果在某些特殊情况下会出现状态混乱。改用状态机后，游戏逻辑变得清晰很多。状态机不仅让代码更容易维护，也大大减少了出错的可能。

模块化设计的重要性也是这次项目给我的重要启示。我将整个系统分为多个功能模块：VGA 驱动、显示控制、移动控制、游戏控制和技能控制。每个模块负责特定的功能，通过清晰的接口相互连接。这种设计方法不仅让开发过程更加条理分明，也让调试变得更加容易。当出现问题时，我可以快速定位到具体是哪个模块出了问题。

参数化设计是另一个让我印象深刻的技巧。通过合理使用参数定义，我可以轻松调整游戏的各种属性，比如移动速度、显示分辨率、技能持续时间等。这种设计方法大大提高了代码的可复用性和可维护性。

在处理按键输入时，我学会了防抖动处理的重要性。最初的设计中，按键输

入经常出现抖动现象，导致玩家角色移动不稳定。通过添加适当的延时和状态检测，最终实现了流畅的控制体验。这

说实话，这个项目完成后我特别有成就感。看着自己设计的小游戏在显示器上运行，按键操作流畅，画面稳定，这种感觉真的很棒。虽然和市面上的游戏比起来还很简陋，但这确实是我从零开始，一行一行代码写出来的作品。

这次项目也让我对 FPGA 开发有了更深的认识。相比软件开发，FPGA 开发需要更多地考虑硬件层面的问题，比如时序约束、资源利用、功耗控制等。

我觉得 FPGA 真是个有趣的平台，它让我们能够直观地理解硬件是如何工作的。虽然调试过程真的很痛苦（尤其是那些莫名其妙的时序错误），但当你真正理解了原理后，就会发现硬件设计其实也有它的逻辑美。这次项目不仅提升了我的硬件设计能力，也让我对数字系统有了更深的理解。

这次项目虽然结束了，但我相信这只是我 FPGA 开发之路的开始。未来我还想尝试实现更复杂的功能，比如添加更多的游戏元素，实现更炫酷的显示效果，甚至尝试一些基于 FPGA 的图像处理项目。

一些建议：

- 1.一定要画好详细的框图再动手写代码，这样可以避免后期的大幅修改
- 2.调试时多用示波器观察信号，它能帮你发现很多肉眼看不到的问题
- 3.遇到问题时要耐心，很多看似神秘的 bug，其实都是时序问题导致的
- 4.善用仿真工具，它能帮你提前发现很多潜在的问题
- 5.做好备份！！！每次成功实现一个功能就更新一遍版本（类似于代码开发中的 git 库上传）这非常重要！指不定哪里搞错了就复原不了了，这时候有备份的话还可以回退版本。

九、附录

（该部分放 verilog 代码，包括所有设计文件和测试文件）

```
module top_vga_move
(
    input wire clk,                // 系统时钟 (100MHz)
    input wire rst_n,              // 低电平复位信号
    input wire key_up,             // 向上移动按键
    input wire key_down,          // 向下移动按键
    input wire key_left,          // 向左移动按键
    input wire key_right,         // 向右移动按键
    input wire skill_key,         // 技能触发按键
    output wire hsync,            // VGA 水平同步信号
    output wire vsync,            // VGA 垂直同步信号
    output wire [3:0] R,          // VGA 红色分量
    output wire [3:0] G,          // VGA 绿色分量
    output wire [3:0] B,          // VGA 蓝色分量
    output wire game_over,        // 游戏结束信号
    output wire game_win,         // 游戏胜利信号
    output wire audio_out,        // 音频输出

```

```

        output wire audio_sd                                // 音频使能
    );

    // 内部信号声明
    wire clk_65M;                                           // 65MHz VGA 时钟
    wire locked;                                           // PLL 锁定信号
    wire frame_flag;                                       // 帧同步标志
    wire [3:0] display_mode;                               // 显示模式控制
    wire [10:0] player_x, player_y;                       // 玩家位置
    wire [10:0] block_x, block_y;                         // 方块位置
    wire [7:0] block_speed;                                // 方块移动速度
    wire hit_boundary;                                     // 边界碰撞信号
    wire is_immune;                                        // 免疫状态
    wire [15:0] game_timer;                                // 游戏计时器
    wire [6:0] skill_segments;                             // 技能剩余次数显示
    wire skill_immune;                                     // 技能免疫状态
    wire game_win_internal;                                // 内部游戏胜利信号
    // 实例化"羊"移动模块
    move_sheep move_sheep(
        .clk(clk_65M),
        .reset_n(rst_n & locked),
        .key_up(key_up),
        .key_down(key_down),
        .key_left(key_left),
        .key_right(key_right),
        .game_over(game_over),
        .frame_flag(frame_flag),
        .player_x_out(player_x),
        .player_y_out(player_y)
    );
    // 实例化 Clocking Wizard 模块
    clk_wiz_0 inst_clk_wiz (
        .clk_in1(clk),
        .reset(~rst_n),
        .clk_out1(clk_65M),
        .locked(locked)
    );
    // 实例化音乐模块
    mxb_music inst_music (
        .clk(clk),
        .collision(game_over), // 连接碰撞信号
        .musicdata(audio_out),
        .music_sd(audio_sd)
    );

```

```

// 游戏控制器实例化
game_controller inst_game_controller (
    .clk(clk_65M),
    .reset_n(rst_n & locked),
    .player_x(player_x),
    .player_y(player_y),
    .block_x(block_x),
    .block_y(block_y),
    .game_over(game_over),
    .display_mode(display_mode),
    .block_speed(block_speed),
    .hit_boundary(hit_boundary),
    .is_immune(is_immune),
    .skill_immune(skill_immune),
    .game_timer(game_timer),
    .game_win(game_win_internal)
);

// 实例化技能控制器
skill_controller inst_skill_controller (
    .clk(clk_65M),
    .reset_n(rst_n & locked),
    .skill_trigger(skill_key),
    .game_over(game_over),
    .skill_count(skill_count),
    .skill_immune(skill_immune),
    .skill_segments(skill_segments)
);

// rgb 输出模块例化
mxb_rgb instial_rgb (
    .clk(clk_65M),
    .reset_n(rst_n & locked),
    .display_mode(display_mode),
    .game_over(game_over),
    .game_win(game_win_internal),
    .player_x(player_x),
    .player_y(player_y),
    .hsync(hsync),
    .vsync(vsync),
    .R(R),
    .G(G),
    .B(B),
    .block_x_out(block_x),
    .block_y_out(block_y),
    .block_speed(block_speed),

```

```

        .hit_boundary_out(hit_boundary),
        .is_immune_out(is_immune),
        .skill_segments(skill_segments),
        .skill_immune(skill_immune),
        .timer_value(game_timer),
        .frame_flag_out(frame_flag)
    );
    // 连接游戏胜利输出信号
    assign game_win = game_win_internal;
endmodule

```

```

module skill_controller #(
    parameter CLK_FREQ = 65_000_000,          // 系统时钟频率 (65MHz)
    parameter SKILL_IMMUNE_TIME = CLK_FREQ * 3, // 技能免疫持续时间 (3 秒)
    parameter INITIAL_SKILL_COUNT = 2'd3      // 初始技能次数
)(
    input wire clk,                          // 系统时钟
    input wire reset_n,                      // 低电平复位信号
    input wire skill_trigger,                // 技能触发输入
    input wire game_over,                    // 游戏结束信号

    output reg [1:0] skill_count,            // 剩余技能次数
    output reg skill_immune,                 // 技能免疫状态
    output wire [6:0] skill_segments        // 七段数码管显示信号
);

// 技能计时器
reg [31:0] skill_timer;
// 7 段数码管显示逻辑
function [6:0] get_segments;
    input [3:0] digit;
    begin
        case(digit)
            4'd0: get_segments = 7'b1110111; // 0
            4'd1: get_segments = 7'b0010010; // 1
            4'd2: get_segments = 7'b1011101; // 2
            4'd3: get_segments = 7'b1011011; // 3
            default: get_segments = 7'b0000000;
        endcase
    end
endfunction

// 技能控制逻辑
always @(posedge clk or negedge reset_n) begin
    if (!reset_n) begin

```

```

        skill_count <= INITIAL_SKILL_COUNT;
        skill_timer <= 0;
        skill_immune <= 0;
    end
    else if (!game_over) begin
        // 技能计时器逻辑
        if (skill_timer > 0) begin
            skill_timer <= skill_timer - 1;
            skill_immune <= 1;
        end
        else begin
            skill_immune <= 0;
        end
        // 技能触发逻辑
        if (skill_trigger && skill_count > 0 && !skill_immune) begin
            skill_count <= skill_count - 1;
            skill_timer <= SKILL_IMMUNE_TIME;
            skill_immune <= 1;
        end
    end
end
// 输出7段数码管显示信号
assign skill_segments = get_segments(skill_count);
endmodule

```

```

module mxb_vga_driver
// 参数定义
#(
    parameter H_CNT_BIT_WIDTH = 10'd11,
    parameter V_CNT_BIT_WIDTH = 10'd11,

    parameter H_SYNC = 10'd136,
    parameter H_TOTAL = 11'd1344,
    parameter V_SYNC = 10'd6,
    parameter V_TOTAL = 10'd806,
    parameter HSYNC_POL = 1'b1, // HSYNC 极性, 1 为正极性
    parameter VSYNC_POL = 1'b1 // VSYNC 极性, 1 为正极性
)
// 端口定义
(
    input wire clk,
    input wire reset_n,
    output reg hsync,
    output reg vsync,

```

```

        output reg [H_CNT_BIT_WIDTH-1:0] h_cnt,
        output reg [V_CNT_BIT_WIDTH-1:0] v_cnt
    );
    // 行扫描计数器
    always @ (posedge clk or negedge reset_n) begin
        if(!reset_n || h_cnt == H_TOTAL-1)
            h_cnt <= 0;
        else
            h_cnt <= h_cnt + 1;
    end
    // 行同步信号
    always @ (posedge clk or negedge reset_n) begin
        if(!reset_n)
            hsync <= ~HSYNC_POL;
        else if(h_cnt == H_SYNC-1)
            hsync <= HSYNC_POL;
        else if(h_cnt == H_TOTAL-1)
            hsync <= ~HSYNC_POL;
    end
    // 场扫描计数器
    always @ (posedge clk or negedge reset_n) begin
        if(!reset_n)
            v_cnt <= 0;
        else if(h_cnt == H_TOTAL-1 && v_cnt == V_TOTAL-1) // 一帧结束
            v_cnt <= 0;
        else if(h_cnt == H_TOTAL-1)
            v_cnt <= v_cnt + 1;
    end
    // 场同步信号
    always @ (posedge clk or negedge reset_n) begin
        if(!reset_n)
            vsync <= ~VSYNC_POL;
        else if(h_cnt == H_TOTAL-1 && v_cnt == V_SYNC-1)
            vsync <= VSYNC_POL;
        else if(h_cnt == H_TOTAL-1 && v_cnt == V_TOTAL-1)
            vsync <= ~VSYNC_POL;
    end
endmodule

```

```

module mxb_rgb
// 参数定义
#(
    parameter H_CNT_BIT_WIDTH = 10'd11,
    parameter V_CNT_BIT_WIDTH = 10'd11,

```

```

parameter H_SYNC = 10'd136,
parameter H_BACK = 10'd160,
parameter H_ACTIVE = 11'd1024,
parameter H_FRONT = 10'd24,
parameter H_TOTAL = 11'd1344,
parameter V_SYNC = 10'd6,
parameter V_BACK = 10'd29,
parameter V_ACTIVE = 10'd768,
parameter V_FRONT = 10'd3,
parameter V_TOTAL = 10'd806,
// 方块的长和宽
parameter LENGTH = 150,
parameter WIDTH = 150,
parameter SPEED_INCREMENT = 1, // 每次碰撞后的速度增量
// 添加小人的参数
parameter PLAYER_SIZE = 20, // 小人大小
parameter PLAYER_SPEED = 5, // 小人移动速度
parameter BORDER_MARGIN = 80, // 边界距离
parameter BORDER_WIDTH = 2, // 边界线宽度
parameter CLK_FREQ = 65_000_000, // 时钟频率 65MHz
parameter IMMUNE_SECONDS = 5, // 需要的免疫秒数
parameter IMMUNE_TIME = CLK_FREQ * IMMUNE_SECONDS, // 计算所需的时钟周期数
// Add new parameters for timer display
parameter TIMER_X = 50, // Timer display X position
parameter TIMER_Y = 50, // Timer display Y position
parameter DIGIT_WIDTH = 15, // Width of each digit
parameter DIGIT_HEIGHT = 20, // Height of each digit
parameter DIGIT_SPACING = 5, // Space between digits
parameter SEG_WIDTH = 15, // Width of each segment
parameter SEG_HEIGHT = 20, // Height of each segment
// 添加技能显示位置参数
parameter SKILL_DISPLAY_X = H_ACTIVE - 100, // 技能显示位置 X
parameter SKILL_DISPLAY_Y = V_ACTIVE - 50 // 技能显示位置 Y
)
// 端口定义
(
    input wire clk,
    input wire reset_n,
    output wire hsync,
    output wire vsync,
    output reg [3:0] R,
    output reg [3:0] G,

```

```

output reg [3:0] B,
input wire [3:0] display_mode, // 从游戏控制器接收显示模式
input wire game_over, // 从游戏控制器接收游戏状态
input wire game_win, // 添加游戏胜利输入端口

input wire [10:0] player_x, // 从移动模块接收羊的位置
input wire [10:0] player_y, // 从移动模块接收羊的位置

output wire [H_CNT_BIT_WIDTH-1:0] block_x_out,
output wire [V_CNT_BIT_WIDTH-1:0] block_y_out,
input wire [7:0] block_speed, // 从游戏控制器接收当前速度
output wire hit_boundary_out, // 添加边界碰撞输出信号
output wire is_immune_out, // 添加免疫状态输出信号
// 添加技能显示相关的输入端口
input wire [6:0] skill_segments, // 7 段数码管显示信号
input wire skill_immune, // 添加技能免疫状态输入
// 计时器显示
input wire [15:0] timer_value,
// 在模块端口定义中添加
output wire frame_flag_out
);
reg frame_flag; // 帧结束标志
reg [H_CNT_BIT_WIDTH-1 : 0] x; // 位置变量 x
reg turn_flag_x; // 位置变量 x 翻转标志
reg [V_CNT_BIT_WIDTH-1 : 0] y; // 位置变量 y
reg turn_flag_y; // 位置变量 y 翻转标志
wire [H_CNT_BIT_WIDTH-1 : 0] h_cnt;
wire [V_CNT_BIT_WIDTH-1 : 0] v_cnt;
// 实例化 VGA 驱动
mxb_vga_driver inst_a_vga_driver (
    .clk      (clk),
    .reset_n  (reset_n),
    .hsync    (hsync),
    .vsync    (vsync),
    .h_cnt    (h_cnt),
    .v_cnt    (v_cnt)
);
// 帧结束标志
always @(posedge clk or negedge reset_n)
begin
    if (!reset_n) begin
        frame_flag <= 0;
    end
end

```

```

    else if ((h_cnt == H_TOTAL - 2) && (v_cnt == V_TOTAL - 1)) begin //提前
一个周期结束
        frame_flag <= 1;
    end
    else begin
        frame_flag <= 0;
    end
end
// 简化的移动方块初始化和移动逻辑
always @(posedge clk or negedge reset_n) begin
    if (!reset_n) begin
        x <= H_ACTIVE/2; // 从中间位置开始
        y <= V_ACTIVE/2;
        turn_flag_x <= 0; // 初始向右移动
        turn_flag_y <= 0; // 初始向下移动
    end
    else if (frame_flag) begin
        // x 方向移动逻辑
        if (x >= H_ACTIVE - LENGTH - block_speed && !turn_flag_x) begin
            turn_flag_x <= 1;
            x <= x;
        end
        else if (x <= block_speed && turn_flag_x) begin
            turn_flag_x <= 0;
            x <= x;
        end
        else if (!turn_flag_x) begin
            x <= x + block_speed;
        end
        else begin
            x <= x - block_speed;
        end
        // y 方向移动逻辑
        if (y >= V_ACTIVE - WIDTH - block_speed && !turn_flag_y) begin
            turn_flag_y <= 1;
            y <= y;
        end
        else if (y <= block_speed && turn_flag_y) begin
            turn_flag_y <= 0;
            y <= y;
        end
        else if (!turn_flag_y) begin
            y <= y + block_speed;
        end
    end
end

```

```

        else begin
            y <= y - block_speed;
        end
    end
end
// 在模块中添加碰撞边界检测信号
reg hit_boundary;
always @(posedge clk or negedge reset_n) begin
    if (!reset_n) begin
        hit_boundary <= 0;
    end
    else begin
        // 检测是否碰到边界，使用范围检测
        if (frame_flag && (
            (x >= H_ACTIVE - LENGTH - block_speed && !turn_flag_x) ||
            (x <= block_speed && turn_flag_x) ||
            (y >= V_ACTIVE - WIDTH - block_speed && !turn_flag_y) ||
            (y <= block_speed && turn_flag_y)
        )) begin
            hit_boundary <= 1;
        end
        else begin
            hit_boundary <= 0;
        end
    end
end
// 在信号定义部分添加
reg [31:0] immune_counter; // 使用32位计数器，确保能容纳大数值
reg is_immune;             // 免疫状态标志
// 添加免疫状态控制
always @(posedge clk or negedge reset_n) begin
    if (!reset_n) begin
        immune_counter <= IMMUNE_TIME;
        is_immune <= 1'b1;
    end
    else if (immune_counter > 0) begin
        immune_counter <= immune_counter - 1;
        is_immune <= 1'b1;
    end
    else begin
        is_immune <= 1'b0;
    end
end
// 添加7段数码管显示逻辑函数

```

```

function [6:0] get_segments;
    input [3:0] digit;
    begin
        case(digit)
            4'd0: get_segments = 7'b1110111; // 0
            4'd1: get_segments = 7'b0010010; // 1
            4'd2: get_segments = 7'b1011101; // 2
            4'd3: get_segments = 7'b1011011; // 3
            4'd4: get_segments = 7'b0111010; // 4
            4'd5: get_segments = 7'b1101011; // 5
            4'd6: get_segments = 7'b1101111; // 6
            4'd7: get_segments = 7'b1010010; // 7
            4'd8: get_segments = 7'b1111111; // 8
            4'd9: get_segments = 7'b1111011; // 9
            default: get_segments = 7'b0000000;
        endcase
    end
endfunction

// 在 get_segments 函数后添加
// BCD 转换函数
function [15:0] decimal_to_bcd;
    input [13:0] decimal;
    reg [15:0] bcd;
    integer i;
    begin
        bcd = 0;
        for (i = 0; i < 14; i = i + 1) begin
            if (bcd[3:0] >= 5) bcd[3:0] = bcd[3:0] + 3;
            if (bcd[7:4] >= 5) bcd[7:4] = bcd[7:4] + 3;
            if (bcd[11:8] >= 5) bcd[11:8] = bcd[11:8] + 3;
            if (bcd[15:12] >= 5) bcd[15:12] = bcd[15:12] + 3;
            bcd = {bcd[14:0], decimal[13-i]};
        end
        decimal_to_bcd = bcd;
    end
endfunction

// 在模块顶部添加 wire 声明
wire [15:0] bcd_time;
wire [3:0] digit_pos;
wire [3:0] current_digit;
wire [6:0] digit_segments;
reg [10:0] rel_x; // 添加相对位置寄存器
reg [10:0] rel_y; // 添加相对位置寄存器
// 计算 segments 和 BCD 时间

```

```

assign bcd_time = decimal_to_bcd(timer_value[13:0]);
assign digit_pos = (h_cnt - (H_SYNC + H_BACK + TIMER_X)) / (DIGIT_WIDTH +
DIGIT_SPACING);
assign current_digit = (digit_pos == 0) ? bcd_time[15:12] :
                        (digit_pos == 1) ? bcd_time[11:8] :
                        (digit_pos == 2) ? bcd_time[7:4] :
                        bcd_time[3:0];
assign digit_segments = get_segments(current_digit);
// RGB 输出逻辑
always @(posedge clk or negedge reset_n) begin
    if (!reset_n) begin
        R <= 4'b0000;
        G <= 4'b0000;
        B <= 4'b0000;
    end
    else if ((h_cnt >= H_SYNC + H_BACK) &&
            (h_cnt < H_SYNC + H_BACK + H_ACTIVE) &&
            (v_cnt >= V_SYNC + V_BACK) &&
            (v_cnt < V_SYNC + V_BACK + V_ACTIVE)) begin
        case(display_mode)
            4'd0: begin // 正常游戏模式
                // 显示边界线 (黄色矩形)
                if ((h_cnt >= (H_SYNC + H_BACK + BORDER_MARGIN) &&
                    h_cnt < (H_SYNC + H_BACK + H_ACTIVE - BORDER_MARGIN) &&
                    ((v_cnt >= (V_SYNC + V_BACK + BORDER_MARGIN) &&
                    v_cnt < (V_SYNC + V_BACK + BORDER_MARGIN + BORDER_WIDTH))
||
                    (v_cnt >= (V_SYNC + V_BACK + V_ACTIVE - BORDER_MARGIN -
BORDER_WIDTH) &&
                    v_cnt < (V_SYNC + V_BACK + V_ACTIVE - BORDER_MARGIN))))
||
                    (v_cnt >= (V_SYNC + V_BACK + BORDER_MARGIN) &&
                    v_cnt < (V_SYNC + V_BACK + V_ACTIVE - BORDER_MARGIN) &&
                    ((h_cnt >= (H_SYNC + H_BACK + BORDER_MARGIN) &&
                    h_cnt < (H_SYNC + H_BACK + BORDER_MARGIN + BORDER_WIDTH))
||
                    (h_cnt >= (H_SYNC + H_BACK + H_ACTIVE - BORDER_MARGIN -
BORDER_WIDTH) &&
                    h_cnt < (H_SYNC + H_BACK + H_ACTIVE - BORDER_MARGIN))))))
begin
                    R <= 4'b1111;
                    G <= 4'b1111;
                    B <= 4'b0000;
                end
            end
        end
    end
end

```

```

// 在移动方块区域内
else if ((h_cnt >= (H_SYNC + H_BACK + x)) &&
         (h_cnt < (H_SYNC + H_BACK + x + LENGTH)) &&
         (v_cnt >= (V_SYNC + V_BACK + y)) &&
         (v_cnt < (V_SYNC + V_BACK + y + WIDTH))) begin
    R <= 1111; // 高 4 位为红色
    G <= 0000; // 中间 4 位为绿色
    B <= 0000; // 低 4 位为蓝色
end
// 在小人区域内
else if ((h_cnt >= (H_SYNC + H_BACK + player_x)) &&
         (h_cnt < (H_SYNC + H_BACK + player_x + PLAYER_SIZE))
&&
         (v_cnt >= (V_SYNC + V_BACK + player_y)) &&
         (v_cnt < (V_SYNC + V_BACK + player_y + PLAYER_SIZE)))
begin
    if (is_immune) begin
        // 开局免疫状态下闪烁效果（蓝绿色交替）
        if (immune_counter[25]) begin
            R <= 4'b0000;
            G <= 4'b1111;
            B <= 4'b1111;
        end else begin
            R <= 4'b0000;
            G <= 4'b0000;
            B <= 4'b1111;
        end
    end
    else if (skill_immune) begin
        // 技能免疫状态下闪烁效果（紫色交替）
        if (h_cnt[4]) begin // 使用不同的计数器来产生不同的闪烁效果
            R <= 4'b1111;
            G <= 4'b1111;
            B <= 4'b0000;
        end else begin
            R <= 4'b1111;
            G <= 4'b0000;
            B <= 4'b1111;
        end
    end
    else begin
        // 正常状态下显示绿色
        R <= 4'b0000;

```

```

        G <= 4'b1111;
        B <= 4'b0000;

    end

    end

    // 显示时间
    else if ((h_cnt >= (H_SYNC + H_BACK + TIMER_X)) &&
        (h_cnt < (H_SYNC + H_BACK + TIMER_X + DIGIT_WIDTH * 4
+ DIGIT_SPACING * 3)) &&
        (v_cnt >= (V_SYNC + V_BACK + TIMER_Y)) &&
        (v_cnt < (V_SYNC + V_BACK + TIMER_Y + DIGIT_HEIGHT)))
begin

    rel_x = (h_cnt - (H_SYNC + H_BACK + TIMER_X)) % (DIGIT_WIDTH
+ DIGIT_SPACING);
    rel_y = v_cnt - (V_SYNC + V_BACK + TIMER_Y);

    if (digit_pos < 4) begin
        if ((digit_segments[6] && rel_y < 2 && rel_x >= 2 && rel_x
< DIGIT_WIDTH-2) ||
            // 顶段
            (digit_segments[5] && rel_y >= 2 && rel_y <
DIGIT_HEIGHT/2-1 && rel_x < 2) ||
            // 左上段
            (digit_segments[4] && rel_y >= 2 && rel_y <
DIGIT_HEIGHT/2-1 && rel_x >= DIGIT_WIDTH-2) ||
            // 右上段
            (digit_segments[3] && rel_y >= DIGIT_HEIGHT/2-1 &&
rel_y < DIGIT_HEIGHT/2+1 &&
            rel_x >= 2 && rel_x < DIGIT_WIDTH-2)
        ||
            // 中段
            (digit_segments[2] && rel_y >= DIGIT_HEIGHT/2+1 &&
rel_y < DIGIT_HEIGHT-2 && rel_x < 2) ||
            // 左下段
            (digit_segments[1] && rel_y >= DIGIT_HEIGHT/2+1 &&
rel_y < DIGIT_HEIGHT-2 &&
            rel_x >= DIGIT_WIDTH-2)
        ||
            // 右下段
            (digit_segments[0] && rel_y >= DIGIT_HEIGHT-2 &&
rel_x >= 2 && rel_x < DIGIT_WIDTH-2)) begin // 底段
            R <= 4'b1111;
            G <= 4'b1111;
            B <= 4'b1111;
        end else begin
            R <= 4'b0000;
            G <= 4'b0000;
            B <= 4'b0000;
        end
    end else begin
        R <= 4'b0000;
        G <= 4'b0000;
        B <= 4'b0000;
    end
end else begin

```

```

        R <= 4'b0000;
        G <= 4'b0000;
        B <= 4'b0000;
    end
end
// 显示技能剩余次数
else if ((h_cnt >= (H_SYNC + H_BACK + SKILL_DISPLAY_X)) &&
        (h_cnt < (H_SYNC + H_BACK + SKILL_DISPLAY_X +
SEG_WIDTH))) &&
        (v_cnt >= (V_SYNC + V_BACK + SKILL_DISPLAY_Y)) &&
        (v_cnt < (V_SYNC + V_BACK + SKILL_DISPLAY_Y +
SEG_HEIGHT))) begin

    // 计算相对位置
    rel_x = h_cnt - (H_SYNC + H_BACK + SKILL_DISPLAY_X);
    rel_y = v_cnt - (V_SYNC + V_BACK + SKILL_DISPLAY_Y);

    // 显示数字段
    if ((skill_segments[6] && rel_y < 2 && rel_x >= 2 && rel_x
< SEG_WIDTH-2) || // 顶段
        (skill_segments[5] && rel_y >= 2 && rel_y <
SEG_HEIGHT/2-1 && rel_x < 2) || // 左上段
        (skill_segments[4] && rel_y >= 2 && rel_y <
SEG_HEIGHT/2-1 && rel_x >= SEG_WIDTH-2) || // 右上段
        (skill_segments[3] && rel_y >= SEG_HEIGHT/2-1 && rel_y
< SEG_HEIGHT/2+1 &&
        rel_x >= 2 && rel_x < SEG_WIDTH-2)
|| // 中段
        (skill_segments[2] && rel_y >= SEG_HEIGHT/2+1 && rel_y
< SEG_HEIGHT-2 && rel_x < 2) || // 左下段
        (skill_segments[1] && rel_y >= SEG_HEIGHT/2+1 && rel_y
< SEG_HEIGHT-2 &&
        rel_x >= SEG_WIDTH-2)
|| // 右下段
        (skill_segments[0] && rel_y >= SEG_HEIGHT-2 && rel_x >=
2 && rel_x < SEG_WIDTH-2)) begin // 底段
        R <= 4'b1111;
        G <= 4'b1111;
        B <= 4'b0000;
    end
else begin
        R <= 4'b0000;
        G <= 4'b0000;
        B <= 4'b0000;
    end
end

```

```

        end
    end
    // 其他区域
    else begin
        R <= 4'b0000;
        G <= 4'b0000;
        B <= 4'b0000;
    end
end
end
4'd1: begin // 游戏结束红条纹
    R <= v_cnt[4] ? 4'b1111 : 4'b0000;
    G <= 4'b0000;
    B <= 4'b0000;
end
4'd2: begin // 游戏胜利全绿色
    R <= 4'b0000;
    G <= 4'b1111;
    B <= 4'b0000;
end
default: begin
    R <= 4'b0000;
    G <= 4'b0000;
    B <= 4'b0000;
end
endcase
end
else begin
    R <= 4'b0000;
    G <= 4'b0000;
    B <= 4'b0000;
end
end
end
assign player_x_out = player_x;
assign player_y_out = player_y;
assign block_x_out = x;
assign block_y_out = y;
// 添加输出端口用于传递边界碰撞信号
assign hit_boundary_out = hit_boundary;
// 在模块末尾添加 is_immune 输出连接
assign is_immune_out = is_immune;
// 在模块内部连接
assign frame_flag_out = frame_flag;
endmodule

```

```

module game_controller
#(
    parameter H_CNT_BIT_WIDTH = 10'd11,    // 水平计数器位宽
    parameter V_CNT_BIT_WIDTH = 10'd11,    // 垂直计数器位宽

    parameter LENGTH = 150,                // 移动方块长度
    parameter WIDTH = 150,                 // 移动方块宽度
    parameter PLAYER_SIZE = 20,            // 玩家角色大小
    parameter BASE_SPEED = 1,              // 移动方块基础速度
    parameter SPEED_INCREMENT = 1,         // 速度增量
    parameter MAX_SPEED = 15,              // 最大速度限制
    parameter SKILL_IMMUNE_TIME = 65_000_000 * 3, // 技能免疫持续时间 (3 秒)
    parameter WIN_TIME = 30                // 游戏通关时间 (30 秒)
)
(
    input wire clk,                        // 系统时钟
    input wire reset_n,                   // 低电平复位信号

    // 位置信号
    input wire [H_CNT_BIT_WIDTH-1:0] player_x,    // 玩家X 坐标
    input wire [V_CNT_BIT_WIDTH-1:0] player_y,    // 玩家Y 坐标
    input wire [H_CNT_BIT_WIDTH-1:0] block_x,      // 方块X 坐标
    input wire [V_CNT_BIT_WIDTH-1:0] block_y,      // 方块Y 坐标
    input wire hit_boundary,                      // 方块碰到边界信号
    input wire is_immune,                         // 开局免疫状态
    input wire skill_immune,                      // 技能免疫状态

    // 输出信号
    output reg game_over,                        // 游戏结束信号
    output reg [3:0] display_mode,               // 显示模式控制
    output reg [7:0] block_speed,                // 方块移动速度
    output reg [15:0] game_timer,                // 游戏计时器
    output reg game_win                          // 游戏胜利信号
);

// 碰撞检测信号
wire collision;
assign collision = (player_x + PLAYER_SIZE > block_x) &&
    (player_x < block_x + LENGTH) &&
    (player_y + PLAYER_SIZE > block_y) &&
    (player_y < block_y + WIDTH);

reg hit_boundary_reg;                        // 边界碰撞寄存器
reg [31:0] skill_timer;                     // 技能计时器
reg [25:0] timer_counter;                  // 游戏时间计数器

```

```

// 主状态控制逻辑
always @(posedge clk or negedge reset_n) begin
    if (!reset_n) begin
        timer_counter <= 0;
        game_timer <= 0;
        game_over <= 0;
        display_mode <= 4'd0;
        block_speed <= BASE_SPEED;
        hit_boundary_reg <= 0;
        skill_timer <= 0;
        game_win <= 0;
    end
    else if (!game_over && !game_win) begin
        if (timer_counter >= 65_000_000 - 1) begin
            timer_counter <= 0;
            game_timer <= game_timer + 1;

            if (game_timer >= WIN_TIME - 1) begin
                game_win <= 1;
                display_mode <= 4'd2;
            end
        end
        else begin
            timer_counter <= timer_counter + 1;
        end

        hit_boundary_reg <= hit_boundary;

        if (skill_timer > 0) begin
            skill_timer <= skill_timer - 1;
        end
        if (collision && !is_immune && !skill_immune) begin
            game_over <= 1;
            display_mode <= 4'd1;
        end
        if (hit_boundary && !hit_boundary_reg) begin
            if (block_speed < MAX_SPEED) begin
                block_speed <= block_speed + SPEED_INCREMENT;
            end
        end
    end
end
endmodule

```

```

module move_sheep
// 参数定义
#(
    parameter H_CNT_BIT_WIDTH = 10'd11, // 水平计数器位宽
    parameter V_CNT_BIT_WIDTH = 10'd11, // 垂直计数器位宽

    parameter H_SYNC = 10'd136, // 水平同步信号宽度
    parameter H_BACK = 10'd160, // 水平后沿
    parameter H_ACTIVE = 11'd1024, // 水平有效显示区域
    parameter H_FRONT = 10'd24, // 水平前沿
    parameter H_TOTAL = 11'd1344, // 水平总像素数
    parameter V_SYNC = 10'd6, // 垂直同步信号宽度
    parameter V_BACK = 10'd29, // 垂直后沿
    parameter V_ACTIVE = 10'd768, // 垂直有效显示区域
    parameter V_FRONT = 10'd3, // 垂直前沿
    parameter V_TOTAL = 10'd806, // 垂直总像素数
    parameter PLAYER_SIZE = 20, // 玩家角色大小
    parameter PLAYER_SPEED = 5, // 玩家移动速度
    parameter BORDER_MARGIN = 80, // 边界距离
    parameter BORDER_WIDTH = 2 // 边界线宽度
)
(
    input wire clk, // 系统时钟
    input wire reset_n, // 低电平复位信号
    input wire key_up, // 向上移动按键
    input wire key_down, // 向下移动按键
    input wire key_left, // 向左移动按键
    input wire key_right, // 向右移动按键
    input wire game_over, // 游戏结束信号
    input wire frame_flag, // 帧同步标志
    output wire [10:0] player_x_out, // 玩家X坐标输出
    output wire [10:0] player_y_out // 玩家Y坐标输出
);
// 玩家位置寄存器
reg [H_CNT_BIT_WIDTH-1:0] player_x;
reg [V_CNT_BIT_WIDTH-1:0] player_y;
// 玩家移动控制
always @(posedge clk or negedge reset_n) begin
    if (!reset_n) begin
        player_x <= H_ACTIVE/2; // 初始化到屏幕中心
        player_y <= V_ACTIVE/2;
    end
    else if (!game_over && frame_flag) begin
        // 上下移动, 限制在边界内
    end
end

```

```

        if (key_up && player_y > BORDER_MARGIN + PLAYER_SPEED)
            player_y <= player_y - PLAYER_SPEED;
        else if (key_down && player_y < V_ACTIVE - BORDER_MARGIN - PLAYER_SIZE
- PLAYER_SPEED)
            player_y <= player_y + PLAYER_SPEED;

        // 左右移动, 限制在边界内
        if (key_left && player_x > BORDER_MARGIN + PLAYER_SPEED)
            player_x <= player_x - PLAYER_SPEED;
        else if (key_right && player_x < H_ACTIVE - BORDER_MARGIN - PLAYER_SIZE
- PLAYER_SPEED)
            player_x <= player_x + PLAYER_SPEED;
    end
end
// 输出信号赋值
assign player_x_out = player_x;
assign player_y_out = player_y;
endmodule

```